



Criminal Network Analysis System

Technical Stack, System Architecture and Feature Design

AI-powered investigation platform using NLP, graph analytics and interactive network visualization

Prototype Architecture Document

1. Executive Summary

The Criminal Network Analysis System is a web-based investigative intelligence platform designed to convert unstructured crime-related reports into structured criminal records and relationship graphs. **The system combines JWT-based authentication, spaCy NLP, PostgreSQL, Neo4j, Cytoscape.js, map-based location views and AI-generated investigative insights.**

The central idea is to allow an investigator to move from raw reports to an interactive intelligence view in a few steps:

- Upload a police report, FIR, intelligence report or related document.
- Use NLP to extract criminals, locations and relationships such as TALKED_TO, LAST_SEEN_AT and ASSOCIATED_WITH.
- Store structured records in PostgreSQL and graph relationships in Neo4j.
- Search criminals by crime category or name.
- Open an individual criminal profile to view location information and a Cytoscape.js relationship graph.
- Navigate from one criminal to related criminals by clicking graph nodes.
- Click a criminal or relationship edge to request AI-generated investigative insight.

Important design principle: AI is used to summarize and interpret available evidence and graph findings. It should not determine guilt or make legal conclusions. The output should be presented as investigative assistance, with links back to source data where possible.

2. System Objectives

- Transform unstructured criminal reports into structured, searchable records.
- Build graph-based relationship maps between criminals, locations and other entities.
- Enable category-based browsing and direct criminal search.
- Support interactive exploration of criminal networks and geographic activity.
- Provide investigator-friendly AI summaries of individuals and relationships.
- Keep the architecture modular enough for future CDR, transaction, blockchain and anomaly-detection modules.

3. Technology Stack

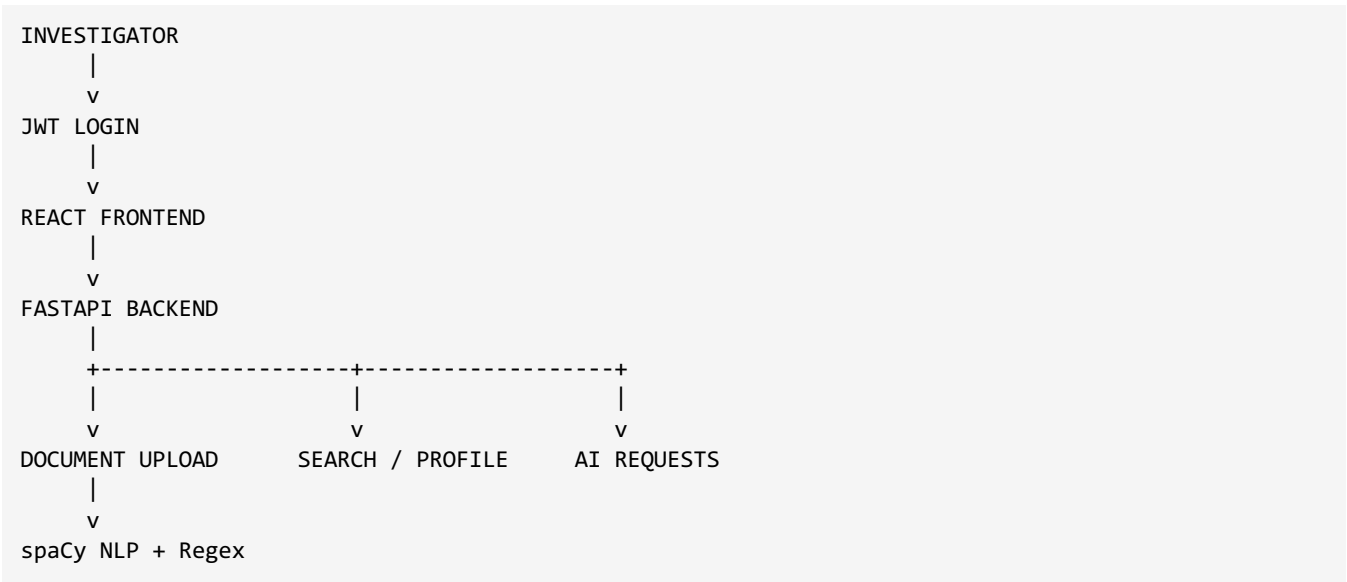
The MVP stack is deliberately small enough to build within two weeks while still demonstrating a complete end-to-end architecture.

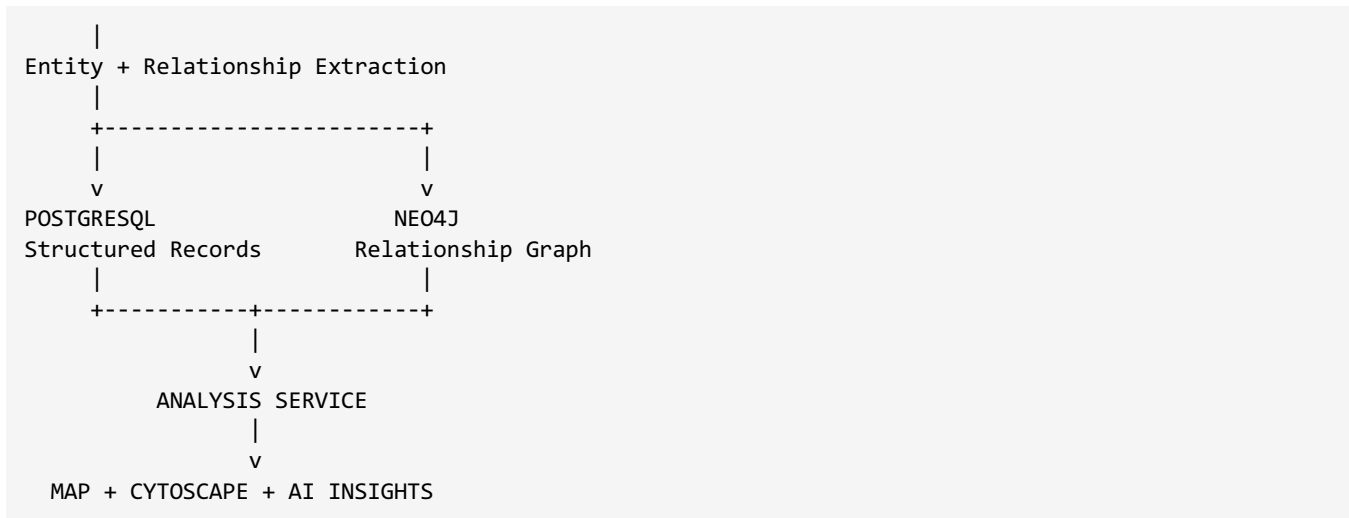
Layer	Technology	Role in the System	Why It Fits the MVP
Frontend	React + Vite	Investigator dashboard, search, profiles and navigation	Fast to develop and easy to integrate with REST APIs
Graph UI	Cytoscape.js	Interactive criminal relationship graph	Designed for node-edge networks and click interactions
Backend	FastAPI	REST APIs, authentication,	Python-native and integrates well with

Layer	Technology	Role in the System	Why It Fits the MVP
		orchestration and business logic	NLP/AI modules
Authentication	JWT	Protects login and authenticated APIs	Simple, stateless and sufficient for an MVP
Structured Database	PostgreSQL	Criminals, cases, documents, categories, locations and relation metadata	Reliable relational storage and strong querying
Graph Database	Neo4j	Nodes, relationships and graph traversal	Natural fit for criminal-network analysis
NLP	spaCy and regex	Entity extraction and basic relation parsing	Works locally and avoids custom model training
Graph Analytics	Neo4j queries	Shortest paths, connected entities and basic centrality	Enough for useful network intelligence
AI Analysis	LLM API	Human-readable insight on criminals and relationships	Adds explanation without replacing deterministic graph analysis

4. High-Level System Architecture

The system is divided into six logical layers: authentication, ingestion, NLP processing, persistence, analytics and presentation.





4.1 Why PostgreSQL and Neo4j Are Both Needed

PostgreSQL stores the factual and tabular side of the investigation: criminal profiles, case metadata, document references, crime categories, known locations and relationship metadata.

Neo4j stores the network side: who is connected to whom, the type of relationship, paths between entities and graph-centric queries.

The NLP/data-processing service should write to both databases. Neo4j does not need to depend on PostgreSQL as its only data source.

5. End-to-End Data Flow

1. User Authentication: The investigator logs in with username/email and password. FastAPI validates credentials and returns a JWT access token. The frontend includes this token in protected API calls.

2. Document Upload: The investigator uploads a supported report such as PDF, DOCX or text. FastAPI stores document metadata and passes extracted text to the NLP service.

3. NLP Processing: spaCy identifies named entities such as PERSON, LOCATION, ORGANIZATION and DATE. Regular expressions can extract phone numbers, vehicle numbers and other structured patterns. Simple rule-based relation extraction identifies relationships such as TALKED_TO or LAST_SEEN_AT.

4. Persistence: Criminal profiles and structured metadata are stored in PostgreSQL. Criminal-to-criminal and criminal-to-location relations are inserted into Neo4j.

5. Search and Retrieval: The frontend requests crime categories, criminal lists or individual criminal profiles through FastAPI APIs.

6. Graph and Location View: Neo4j returns a relationship subgraph for the selected criminal. PostgreSQL/Neo4j provide known location data. The frontend renders the graph with Cytoscape.js.

7. AI Insight: When the investigator requests analysis, FastAPI collects relevant structured profile data, relationship data and graph findings, then sends a concise context package to the AI model. The result is returned as an investigator-friendly explanation.

6. Feature Design

6.1 JWT Authentication

- Login endpoint validates user credentials.
- Passwords are stored only as secure hashes, never as plaintext.
- A JWT access token is returned after successful login.
- Protected APIs require Authorization: Bearer <token>.
- For the MVP, complex refresh-token rotation, SSO and MFA can be left as future scope.

6.2 Document Upload and NLP Extraction

Example report text:

“Rahul Sharma was last seen near Pune Railway Station and frequently communicated with Amit Singh.”

Expected extraction:

```
Rahul Sharma -> PERSON
Pune Railway Station -> LOCATION
Amit Singh -> PERSON
Rahul Sharma -> LAST_SEEN_AT -> Pune Railway Station
Rahul Sharma -> TALKED_TO -> Amit Singh
```

For the two-week MVP, use pre-defined relationship rules rather than training a custom relation-extraction model.

6.3 Crime Category Browsing

The investigator can select a crime category and view all criminals assigned to that category. Example categories may include cybercrime, financial fraud, organized crime, narcotics or smuggling, depending on the demonstration dataset.

```
GET /crime-categories/{category}/criminals
```

6.4 Criminal Search

A search bar allows direct lookup by criminal name or identifier. Selecting a result opens the criminal profile page.

```
GET /criminals/search?q=Rahul
```

6.5 Criminal Profile Page

The profile page should be the central investigation workspace. A recommended layout is:

```
+-----+
| Rahul Sharma | Crime Category | Basic Profile          |
+-----+-----+-----+
| LOCATION / MAP           | RELATIONSHIP GRAPH       |
| Last seen                | Cytoscape.js             |
| Known locations          | Clickable nodes and edges |
+-----+-----+-----+
| AI ANALYSIS BUTTON / INSIGHT PANEL |
+-----+-----+-----+
```

6.6 Interactive Relationship Graph

- The selected criminal appears as the focus node.
- Connected criminal nodes are loaded from Neo4j.
- Edges are labelled by relationship type such as TALKED_TO, LAST_SEEN_AT, MET or ASSOCIATED_WITH.
- Clicking a related criminal node navigates to that person's profile.
- Clicking a relationship edge opens relation-specific details and an AI analysis action.
- Graph depth should be limited for performance, for example one or two hops by default.

6.7 Location and Pan-India View

Use two distinct visualization modes instead of forcing geographic behavior into a pure network layout:

- Network View: Cytoscape.js displays persons and their relationship structure.
- India Map View: Leaflet/OpenStreetMap displays criminal locations, last-seen points and related persons geographically.

The investigator can switch between the views depending on whether the question is relationship-centric or geography-centric.

6.8 AI Analysis of a Criminal

When the investigator clicks "AI Analysis", the backend creates a compact evidence package containing:

- Criminal profile and category
- Known and last-seen locations
- Direct relationships from Neo4j
- Shared cases/locations if available
- Basic graph indicators such as number of connections or bridging position

The AI converts this into a readable investigative summary. The UI should label it as AI-generated investigative insight and avoid presenting it as a legal conclusion.

6.9 AI Analysis of a Relationship

When the investigator clicks an edge such as Rahul -- TALKED_TO --> Amit, the backend gathers relation-specific evidence, including shared documents, shared locations and network paths. The AI then explains why the connection may be relevant.

7. Data Model

7.1 PostgreSQL Core Tables

users: id, name, email, password_hash, role

criminals: id, name, age/dob (optional), crime_category_id, description, risk_label (optional)

crime_categories: id, name, description

documents: id, case_id, filename, source_type, uploaded_by, uploaded_at

criminal_locations: criminal_id, location_id, relation_type, observed_at, document_id

7.2 Neo4j Graph Model

```
(:Criminal)-[:TALKED_TO]->(:Criminal)
(:Criminal)-[:MET]->(:Criminal)
```

```
(:Criminal)-[:ASSOCIATED_WITH]->(:Criminal)
(:Criminal)-[:LAST_SEEN_AT]->(:Location)
(:Criminal)-[:OPERATES_FROM]->(:Location)
```

8. Simplified API Design

The MVP does not need dozens of endpoints. The following set covers the primary workflow.

Endpoint	Method	Purpose
/auth/login	POST	Authenticate user and issue JWT
/documents/upload	POST	Upload and process a report using spaCy
/crime-categories	GET	List crime categories
/crime-categories/{category}/criminals	GET	List criminals in a category
/criminals/search	GET	Search criminal by name
/criminals/{id}	GET	Get criminal profile
/criminals/{id}/locations	GET	Get known/last-seen locations
/criminals/{id}/network	GET	Return graph nodes and edges
/ai/criminal/{id}	POST	Generate criminal insight
/ai/relation/{relation_id}	POST	Generate relationship insight

9. Backend Module Responsibilities

Authentication Module: JWT creation/validation, password verification, protected-route dependency.

Upload Module: Receives documents, extracts text and stores document metadata.

NLP Module: Runs spaCy, regex and rule-based relation extraction.

PostgreSQL Service: Stores and queries structured criminal/case/location data.

Neo4j Service: Creates/updates graph nodes and relationships; returns subgraphs and paths.

AI Service: Builds constrained prompts from retrieved facts and returns explanatory analysis.

Frontend Integration Layer: Returns predictable JSON structures for search, profile, graph and map components.

11. Recommended Demo Scenario

1. Investigator logs in using JWT authentication.
2. Investigator uploads a sample report containing two criminals, one location and one relationship.
3. The system extracts entities and relations automatically.
4. The investigator opens a crime category and selects one criminal.
5. The profile displays last-seen/known locations and the Cytoscape network.
6. The investigator clicks a related criminal node and navigates to that criminal's profile.
7. The investigator returns and clicks the relation edge.
8. AI explains the relevance of that relationship using retrieved evidence.
9. The investigator requests an AI summary of the selected criminal and network.

12. Future Extensions

- CDR and transaction ingestion pipelines.
- Graph centrality and community-detection dashboards.
- Entity resolution to merge duplicate people across multiple reports.
- Evidence chain-of-custody and tamper verification using blockchain hashes.
- Role-based access controls for investigators, analysts and administrators.
- Source-level citations in AI responses and confidence scoring for NLP-extracted relations.
- Advanced anomaly detection for calls, transactions and cross-location activity.

13. Summary

Core pipeline: Authenticate -> Upload -> NLP -> PostgreSQL + Neo4j -> Search/Profile -> Graph/Map -> AI Insight.

This architecture is technically coherent, understandable in a project evaluation, and feasible as a two-week prototype if the team focuses first on a single reliable workflow and uses deterministic NLP/graph logic before adding optional intelligence features.